

1) Veriler temiz mi? Temiz verinin ne anlama geldiğini açıklayın. İçinde yinelenen kayıtlar var mı? Varsa, bu tekrarları giderin.

- Kodumda duplicated().sum() fonksiyonunu kullanarak yinelenen kayıt var mı baktım o çıktı yani yinelenen veri yok ve Portekizceden çeviri yaptım yani verileri elimizden geldiğince temiz ve düzenli yaptım.
- Ancak ben yine de her ihtimale karşı gelecekte oluşabilecek hataları engellemek için drop_duplicates() komutunu kullandım.
- Temiz veri; kopyalardan arınmış, eksik değerleri mantıksal olarak doldurulmuş veya elenmiş, format bütünlüğü sağlanmış veridir.

```
# 1. Tablolarda tamamen aynı olan hatalı çift kayıtlar var mı bakıyoruz
print("Siparişler tablosundaki mükerrer satır sayısı:", orders.duplicated().sum())
print("Sipariş detayları tablosundaki mükerrer satır sayısı:", order_items.duplicated().sum())

# 2. Eğer varsa bu mükerrer kayıtları temizliyoruz
orders = orders.drop_duplicates()
order_items = order_items.drop_duplicates()

print("Mükerrer veri kontrolü ve temizliği tamamlandı!")
```

[4] ✓ 0.4s Python

... Siparişler tablosundaki mükerrer satır sayısı: 0
Sipariş detayları tablosundaki mükerrer satır sayısı: 0
Mükerrer veri kontrolü ve temizliği tamamlandı!

```
# 1. Ürünler ve çeviri dosyalarını bilgisayardaki o klasörden okutuyoruz
products = pd.read_csv(os.path.join(klasor, "olist_products_dataset.csv"))
translation = pd.read_csv(os.path.join(klasor, "product_category_name_translation.csv"))

# 2. Portekizce isimler ile İngilizce karşılıklarını birleştiriyoruz (Sözlükten bakıp eşleştiriyoruz)
products = pd.merge(products, translation, on="product_category_name", how="left")

# 3. Kafamız karışmasın diye eski Portekizce sütunu silip, yeni İngilizce sütunun adını düzeltiyoruz
products = products.drop(columns=["product_category_name"])
products = products.rename(columns={"product_category_name_english": "product_category_name"})

print("Kategoriler başarıyla İngilizceye çevrildi!")
print("Yeni tablodaki ilk 3 ürünün İngilizce kategorisi:")
print(products["product_category_name"].head(3))
```

[5] ✓ 0.1s Python

... Kategoriler başarıyla İngilizceye çevrildi!
Yeni tablodaki ilk 3 ürünün İngilizce kategorisi:
0 perfumery
1 art
2 sports_leisure
Name: product_category_name, dtype: object

2) Yukarıda verilen girdi verilerine dayanarak, aşağıdaki iş sorularını yanıtlamak için çıktımız nasıl görünmelidir? Bunun gerekçesini açıklayın.

Böyle büyük veri setlerini okuyup sonuç çıkarmak neredeyse imkansız o yüzden biz bu veri setlerini anlamlı hale getirmeye çalışıyoruz .Ben groupby(), sum(), count() gibi fonksiyonları kullanarak verileri basitleştirmeye gruplamaya düzenlemeye çalıştım. Ayrıca her şeyi görsel hale getirmek için grafikler oluşturdum.Böylece şirketlerin daha hızlı karar almasına , zaman ve para kaybetmemesine ve iş gücünü azaltmasına yardımcı olduk .

```
# 1. Toplam ciro ve satılan toplam ürün adedini hesaplıyoruz
toplam_ciro = siparis_detay_raporu["price"].sum()
toplam_urun_adi = len(siparis_detay_raporu)

print("--- GENEL SATIŞ RAPORU ---")
print(f"Şirketin Toplam Ciro: {toplam_ciro:.2f} USD")
print(f"Satılan Toplam Ürün Adedi: {toplam_urun_adi:.} adet")

[7] ✓ 0.0s Python

--- GENEL SATIŞ RAPORU ---
Şirketin Toplam Ciro: 13,591,643.70 USD
Satılan Toplam Ürün Adedi: 112,650 adet

# Kategorilere göre ciroyu (price) topluyoruz, büyüten küçüğe sıralayıp ilk 5 tanesini seçiyoruz
en_cok_kazandiranlar = siparis_detay_raporu.groupby("product_category_name")["price"].sum().sort_values(ascending=False).head(5)

print("--- ŞİRKETE EN ÇOK PARA KAZANDIRAN İLK 5 KATEGORİ ---")
print(en_cok_kazandiranlar)

[8] ✓ 0.0s Python

--- ŞİRKETE EN ÇOK PARA KAZANDIRAN İLK 5 KATEGORİ ---
product_category_name
health_beauty          1258681.34
watches_gifts          1205005.68
bed_bath_table          1036988.68
sports_leisure           988048.97
computers_accessories    911954.32
Name: price, dtype: float64
```

3) Veriyi dönüştürmek için benimsediğiniz yaklaşım ETL mi yoksa ELT mi; neden? dbt hakkında daha fazla bilgi edinin ve açıklayın.

Benim benimsediğim yaklaşım ELT çünkü;

- Ham veriyi doğrudan sisteme yüklediği için ham veri korunur sonrasında ham veriye ihtiyacımız olduğunda kullanabiliriz.
- Verilerde istediğimiz dönüşümü yapmamızın esnekliğini sağlar Portekizceden yaptığım çeviri dönüşümü gibi dönüşümleri kolayca yapmamın sebebidir.

DBT: Elt mimarisinin dönüştürme aşamasına odaklanmış modern bir veri mühendisliği aracıdır. Ham veriyi temiz tablolara dönüştürmek için kullanılır.

4) Mimarinizdeki katmanları ve/veya mantığı açıklayın.

- **Ham veri girişi katmanı :**
Ham veri girişi yaptık.
- **Temizlenmiş ve standartlaştırılmış veri:**
Ham katmandaki veriler temizlendi , biçimlendirildi ve birbiriyle ilişkilendirildi . duplicated().sum() ile mükerrer kayıtların elenmesi, Portekizce kategori isimlerinin İngilizceye çevrilerek standartlaştırılması, sipariş tarihi sütununun (order_purchase_timestamp) string formatından gerçek bir zaman serisi formatına (datetime) dönüştürülmesi bu katmanda gerçekleşmiştir.
- **İş ve analiz katmanı:**
Verilerin kullanıcılar için hazır hale getirildiği son katmandır. groupby() fonksiyonu ile ürettiğimiz "Aylara Göre Toplam Sipariş Sayıları", "En Çok Ciro Getiren İlk 5 Kategori" tabloları ve bu tablolardan çizdirdiğimiz Matplotlib grafikleri projenin bu katmanındadır.

Yıldız şeması mantığı: Ortada merkez ve etrafında boyutlar vardır

Merkez: order_items tablosudur; çünkü fiyat gibi sürekli hesaplanan sayısal verileri tutar.

Etraf: products, customers ve sellers tablolarıdır; çünkü merkezdeki sipariş satırlarına "Hangi şehir?", "Hangi satıcı?" detaylarını beslerler.

5) Yıldız şeması (star schema) oluşturun; olgu (fact) ve boyut (dimension) tablolarını ve bu tabloların sorulara nasıl yanıt verebileceğini açıklayın.

```
# 1. Merkezdeki Fact (Sipariş detayları) tablosunu alıyoruz
# 2. Etrafındaki Dimension (Ürünler ve Müşteriler) tablolarını tek seferde bağlıyoruz

# Önce ürün kategorilerini İngilizceye çevirdiğimiz products tablosuyla birleştiriyoruz
yildiz_tablosu = pd.merge(order_items, products[["product_id", "product_category_name"]], on="product_id", how="left")

# Şimdi sipariş tarihi (year_month) için orders tablosunu bağlıyoruz
yildiz_tablosu = pd.merge(yildiz_tablosu, orders[["order_id", "year_month", "customer_id"]], on="order_id", how="left")

# Son olarak müşteri şehir ve eyaletleri için customers tablosunu bağlıyoruz
yildiz_tablosu = pd.merge(yildiz_tablosu, customers[["customer_id", "customer_city", "customer_state"]], on="customer_id", how="left")

# Tablonun oluşup oluşmadığını kontrol etmek için ilk 3 satırını ekrana yazdırıyoruz
yildiz_tablosu.head(3)
```

order_id	order_item_id	product_id	seller_id	shipping_limit_date	price	freight_value	product_category_name	year_month
00010242fe8c5a6d1ba2dd792cb16214	1	4244733e06e7ecb4970a6e2683c13e61	48436dade18ac8b2bce089ec2a041202	2017-09-19 09:45:35	58.9	13.29	cool_stuff	2017-09
00018f77f2f0320c557190d7a144bdd3	1	e5f2d52b802189ee658865ca93d83a8f	dd7ddc04e1b6c2c614352b383efed2d36	2017-05-03 11:05:13	239.9	19.93	pet_shop	2017-04
000229ec398224ef6ca0657da4fc703e	1	c777355d18b72b67abbeef9df44fd0fd	5b51032eddd242ad84c38acab88f23d	2018-01-18 14:48:30	199.0	17.87	furniture_decor	2018-01

```
# Kurduğumuz yıldız tablosu üzerinden satıcı bazlı ciro analizi yapıyoruz
en_basarili_saticilar = yildiz_tablosu.groupby("seller_id")["price"].sum().sort_values(ascending=False).head(5)

print("--- EN ÇOK CİRO YAPAN İLK 5 SATICI ---")
print(en_basarili_saticilar)
```

seller_id	price
4869f7a5dfa277a7dca6462dcf3b52b2	229472.63
53243585a1d6dc2643021fd1853d8905	222776.05
4a3ca9315b744ce9f8e9374361493884	200472.92
fa1c13f2614d7b5c4749cbc52fedca94	194042.03
7c67e1448b0f6e969d365cea6b018ab	187923.89

Name: price, dtype: float64

Projemizde, ham e-ticaret verilerinin hızlı, optimize ve ölçeklenebilir bir şekilde analiz edilebilmesi için veri ambarı standartlarına uygun bir Yıldız Şeması mimarisi kurulmuştur. Olgu Tablosu: olist_order_items_dataset (Merkez Tablo)

İçerik: Ürün fiyatı (price), kargo ücreti (freight_value) gibi sürekli hesaplanabilir, agregre edilebilir (sum, mean) sayısal verileri ve boyut tablolarına bağlanmayı sağlayan ilişki anahtarlarını (order_id, product_id, seller_id) içerir.

Boyut (Dimension) Tabloları:

olist_products_dataset: Ürünlerin kategorilerini (product_category_name) tutar.

olist_orders_dataset: Siparişlerin ne zaman verildiğini (year_month / zaman serisi) tutar.

olist_customers_dataset: Müşterilerin coğrafi konumlarını (customer_city, customer_state) tutar.

```
import pandas as pd

# Merkezdeki Fact (Sipariş detayları) tablosuna, etraftaki tüm Boyut (Dimension) tabloları bağlanır
# 1. Ürün Boyutu Bağlantısı (Kategoriler)
yildiz_tablosu = pd.merge(order_items, products[["product_id", "product_category_name"]], on="product_id", how="left")

# 2. Sipariş Boyutu Bağlantısı (Zaman Serisi)
yildiz_tablosu = pd.merge(yildiz_tablosu, orders[["order_id", "year_month", "customer_id"]], on="order_id", how="left")

# 3. Müşteri Boyutu Bağlantısı (Coğrafi Konum)
yildiz_tablosu = pd.merge(yildiz_tablosu, customers[["customer_id", "customer_city", "customer_state"]], on="customer_id", how="left")
```

İşletme Sorusu	Bilgi Tablosu (Fact Table)	Boyutlar (Dimensions)
Aylık gelir	olist_order_items_dataset (price)	olist_orders_dataset (order_purchase_timestamp → Yıl-Ay)
Ürün kategorisine göre gelir	olist_order_items_dataset (price)	olist_products_dataset (product_category_name)
En başarılı satıcılar	olist_order_items_dataset (price)	olist_sellers_dataset (seller_id)
Müşteri eyaletine göre satışlar	olist_order_items_dataset (price)	olist_customers_dataset (customer_state)
Eyaletlere göre ortalama teslim süresi	olist_orders_dataset (order_delivered_customer_date - order_purchase_timestamp)	olist_customers_dataset (customer_state)
Ödeme yöntemi trendleri	olist_order_payments_dataset (payment_value)	olist_orders_dataset (order_purchase_timestamp) ve olist_order_payments_dataset (payment_type)
Kategoriye göre ortalama değerlendirme puanı	olist_order_reviews_dataset (review_score)	olist_products_dataset (product_category_name)